

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ ПО ИГРЕ

по дисциплине
«Информатика и программирование»

Студент

гр. БИН-25-2

К.В. Джаманшалов

Преподаватель

А.Д. Шнейдер

Задание

Реализовать прототип консольной текстовой RPG-игры на языке Python. Программа должна работать в интерактивном пошаговом режиме, управляться командами с клавиатуры и предоставлять пользователю базовый игровой опыт в стиле классических roguelike.

Общая идея. Проект представляет собой текстовую RPG, в которой игрок:

- создаёт персонажа с уникальным набором характеристик;
- исследует случайно генерируемое подземелье, состоящее из различных комнат;
- сражается с противниками, получает предметы и опыт;
- управляет инвентарём и экипировкой;
- развивает персонажа, повышая его уровень и характеристики.

Создание персонажа. В начале игры пользователь выбирает расу для своего персонажа (Человек, Лесной эльф, Горный дворф). Каждая раса задаёт уникальные допустимые диапазоны для базовых характеристик, таких как здоровье, сила, ловкость и выносливость. Характеристики генерируются случайно в этих пределах. Дополнительные физические параметры (рост, вес) могут оказывать влияние на игровые механики.

Прогрессия и экипировка. Персонаж получает опыт за победы над врагами, что приводит к повышению уровня. Каждый новый уровень даёт очки для ручного улучшения характеристик. У игрока есть инвентарь для хранения предметов: зелий, валюты, оружия и брони. Экипированные предметы напрямую влияют на характеристики в бою. Управление ресурсами (местом в инвентаре, выбором экипировки) является важной частью геймплея.

Подземелье и встречи. Игровой процесс построен на последовательном прохождении комнат подземелья. На развилках игрок выбирает направление (влево или вправо), каждое из которых ведёт в комнату случайного типа:

- Боевая комната: встреча с противником.
- Комната с сокровищами: награда с возможной ловушкой.
- Комната отдыха: безопасная зона для восстановления и развития.

Сложность (характеристики врагов, ценность наград) прогрессирует с увеличением уровня подземелья.

Боевая система и награды. Сражения происходят в пошаговом режиме. На исход боя влияют характеристики персонажа и врага, экипировка, а также случайные факторы (критический удар, уклонение). Предметы и валюту можно получить, побеждая врагов или находя сундуки.

Содержание

Введение	3
1 Гейм-дизайн.....	4
2 Выполнение.....	5
2.1 Основная программа и главный цикл.....	5
2.2 Создание персонажа	5
2.3 Главное меню игры.....	6
2.4 Исследование комнат подземелья	7
2.5 Боевая система	8
2.6 Система врагов	9
2.7 Система инвентаря и экипировки.....	10
2.8 Система уровней и развития	11
3 Тестирование	13
Заключение	13

Введение

Разработка консольных приложений, в частности игр, является важным этапом в освоении программирования, так как позволяет применить на практике знания об алгоритмах, структурах данных, объектно-ориентированном проектировании и взаимодействии с пользователем.

Целью данной работы была реализация прототипа консольной текстовой ролевой игры (RPG) на языке Python, соответствующего заданным требованиям. Основной акцент делался на создании логичной, расширяемой архитектуры и отработке ключевых игровых механик, таких как создание и развитие персонажа, процедурная генерация контента и тактическая боевая система.

1 Гейм-дизайн

При проектировании игры были заложены следующие ключевые принципы, определившие архитектурные и программные решения:

- Простота и ясность управления: весь игровой процесс управляется через последовательность текстовых меню с числовым вводом, что делает интерфейс доступным и предсказуемым для пользователя.

- Ощущение прогрессии: игрок чувствует рост силы своего персонажа благодаря системе уровней, ручному распределению характеристик и получению более мощной экипировки по мере прохождения.

- Баланс выбора и случайности: благодаря процедурной генерации подземелья, врагов и наград каждая игровая сессия становится уникальной. При этом игрок сохраняет контроль над стратегическими решениями: выбор пути развития персонажа, тактика в бою, управление ресурсами инвентаря.

- Разнообразие игровых стилей: три доступные расы (Человек, Лесной эльф, Горный дворф) имеют различные стартовые параметры и потенциальные пути развития, что поощряет эксперименты и повторные прохождения.

Эти принципы легли в основу проектирования модульной структуры программы и реализации её основных систем.

2 Выполнение

2.1 Основная программа и главный цикл

Основная программа находится в файле `adventure.py` и отвечает за инициализацию игры и запуск главного игрового цикла. Её реализация представлена на рисунке 1.

```

1 from hero import Hero
2
3 def main():
4     player = Hero()
5     player.create_character()
6
7     adventure_continues = True
8     while adventure_continues:
9         if player.game_menu():
10            print("\n" + "=" * 50)
11            print("          ПУТЕШЕСТВИЕ ЗАКОНЧЕНО          ")
12            print("Надеемся увидеть вас снова, искатель приключений!")
13            print("=" * 50)
14            adventure_continues = False
15
16 if __name__ == "__main__":
17     main()
18

```

Рисунок 1 – Листинг основной программы

Программа начинается с импорта класса `Hero` из модуля `hero.py`. В функции `main()` создается экземпляр класса `Hero` с именем `player`. Вызывается метод `create_character()`, который запускает процесс создания персонажа.

Запускается бесконечный цикл `while adventure_continues`, который продолжается до тех пор, пока игрок не завершит игру или не погибнет. На каждой итерации цикла вызывается метод `game_menu()`, который отображает главное меню игры и обрабатывает выбор игрока. Если метод `game_menu()` возвращает `True` (игрок решил покинуть подземелье), цикл прерывается и игра завершается.

2.2 Создание персонажа

Процесс создания персонажа реализован в методе `create_character()`. Код метода показан на рисунке 2.

```

1 def create_character(self):
2     print("=" * 50)
3     print("                СОЗДАНИЕ ГЕРОЯ                ")
4     print("=" * 50)
5
6     self.name = input("Как зовут вашего искателя приключений?\n> ")
7
8     print("\Выберите ваше происхождение:")
9     print("1. Человек сбалансированный()")
10    print("2. Лесной эльф проворный()")
11    print("3. Горный дворф выносливый()")
12
13    heritage_choice = input("\Ваш выбор (1-3): ")
14
15    if heritage_choice == "1":
16        self.heritage = "Человек"
17        self.max_health = randint(85, 105)
18        self.might = randint(7, 11)
19        self.finesse = randint(6, 11)
20        self.resilience = randint(2, 5)
21
22    elif heritage_choice == "2":
23        self.heritage = "Лесной эльф"
24        self.max_health = randint(75, 95)
25        self.might = randint(6, 10)
26        self.finesse = randint(8, 14)
27        self.resilience = randint(2, 4)
28
29    elif heritage_choice == "3":
30        self.heritage = "Горный дворф"
31        self.max_health = randint(105, 135)
32        self.might = randint(8, 13)
33        self.finesse = randint(3, 7)
34        self.resilience = randint(4, 7)
35
36    else:
37        print("Неверный выбор! Создаем человека по умолчанию.")
38        self.heritage = "Человек"
39        self.max_health = randint(85, 105)
40        self.might = randint(7, 11)
41        self.finesse = randint(6, 11)
42        self.resilience = randint(2, 5)
43

```

Рисунок 2 – Листинг создания персонажа

Программа запрашивает имя персонажа у пользователя. Предлагается выбор из трёх рас: Человек, Лесной эльф, Горный дворф. В зависимости от выбора расы генерируются случайные характеристики в заданных диапазонах. Человек имеет сбалансированные характеристики, Лесной эльф отличается повышенной ловкостью, а Горный дворф обладает высоким здоровьем и стойкостью.

После установки характеристик генерируются физические параметры: рост и вес. Персонаж получает стартовое снаряжение в зависимости от расы. Текущее здоровье устанавливается равным максимальному. В завершение вызывается метод для отображения характеристик созданного персонажа.

2.3 Главное меню игры

Метод `game_menu()` управляет основным игровым циклом и предоставляет интерфейс для взаимодействия с игроком. Код представлен на рисунке 3.

```

1 def game_menu(self):
2     if self.current_health <= 0:
3         print("\nВаше путешествие окончено...")
4         return self.game_over()
5
6     self.gain_experience()
7     self.check_dungeon_progress()
8
9     print("\n" + "=" * 50)
10    print(f"    УРОВЕНЬ ПОДЗЕМЕЛЬЯ: {self.dungeon_level}")
11    print(f"    ПРОЙДЕНО КОМНАТ: {self.chambers_cleared}")
12    print("=" * 50)
13
14    choice = input("Ваш выбор: ")
15
16    if choice == "1":
17        self.explore_chamber()
18    elif choice == "2":
19        self.check_inventory()
20    elif choice == "3":
21        self.display_character_sheet()
22    elif choice == "4":
23        self.manage_gear()
24    elif choice == "5":
25        return True
26
27    return False
28

```

Рисунок 3 – Листинг главного меню игры

Проверяется здоровье персонажа. Если оно равно или меньше нуля, игра завершается. Проверяется возможность повышения уровня на основе накопленного опыта. Проверяется прогресс в подземелье для возможного перехода на следующий уровень.

Выводится информационная панель с текущим состоянием персонажа, включая уровень подземелья, количество пройденных комнат, здоровье, количество золота и текущую экипировку. Отображается меню доступных действий.

Обрабатывается выбор пользователя. При выборе исследования новой комнаты вызывается метод `explore_chamber()`. Для просмотра инвентаря используется `check_inventory()`. Просмотр характеристик персонажа осуществляется через `display_character_sheet()`. Управление предметами и экипировкой выполняется методом `manage_gear()`. Возвращается `True`, если игрок решил покинуть подземелье, иначе `False`.

2.4 Исследование комнат подземелья

Метод `explore_chamber()` реализует процедурную генерацию комнат подземелья. Его исполнение приведено на рисунке 4.


```

1 def explore_chamber(self):
2     self.chambers_cleared += 1
3
4     chamber_types = ['боевая комната', 'комната сокровищ', 'комната отдыха']
5     left_chamber = choice(chamber_types)
6     right_chamber = choice(chamber_types)
7
8     print(f"\Вып стоите на развилке подземных ходов...")
9     print(f"Слева: {left_chamber}")
10    print(f"Справа: {right_chamber}")
11
12    direction = input("Направление: ")
13
14    if direction == "1":
15        chosen_path = left_chamber
16    elif direction == "2":
17        chosen_path = right_chamber
18    else:
19        print("Вы заблудились и возвращаетесь назад!")
20        return
21
22    if chosen_path == 'боевая комната':
23        self.combat_encounter()
24    elif chosen_path == 'комната сокровищ':
25        self.treasure_vault()
26    else:
27        self.safe_haven()
28

```

Рисунок 4 – Листинг исследования комнат

Увеличивается счетчик пройденных комнат. Случайным образом выбираются типы комнат для левого и правого путей из трех возможных вариантов: боевая комната, комната сокровищ или комната отдыха.

Игроку предоставляется выбор направления. В зависимости от выбора и типа комнаты вызывается соответствующая функция. Боевая встреча с врагом обрабатывается методом `combat_encounter()`. Комната с сокровищами, где возможна ловушка, управляется через `treasure_vault()`. Комната отдыха для восстановления реализована в методе `safe_haven()`.

Система обеспечивает разнообразие игрового процесса за счёт случайного распределения типов комнат и предоставляет игроку возможность тактического выбора пути в зависимости от текущего состояния персонажа.

2.5 Боевая система

Боевая система реализована в методе `combat_encounter()` и обеспечивает тактический пошаговый бой. Соответствующий код показан на рисунке 5.

```

1 def combat_encounter(self):
2     foe = DungeonCreature(self.dungeon_level, self.chambers_cleared)
3     foe_health, foe_max_health, foe_power, foe_speed, foe_armor, foe_exp, foe_gold = foe.
        display_creature_info()
4
5     foe_current_health = foe_health
6
7     while self.current_health > 0 and foe_current_health > 0:
8         print(f"\Ваше здоровье: {self.current_health}/{self.max_health}")
9         print(f"Здоровье {foe.name}: {foe_current_health}/{foe_max_health}")
10
11        action = input("Выбор: ")
12
13        if action == "1":
14            if foe_speed > self.finesse and randint(1, 20) <= min(foe_speed - self.finesse, 8):
15                print(f"{foe.name} ловко уворачивается!")
16            else:
17                base_damage = self.might
18                min_damage = max(4, base_damage // 2)
19                damage = base_damage - foe_armor
20                if damage < min_damage:
21                    damage = min_damage
22
23                if randint(1, 10) == 1:
24                    damage *= 2
25                print("СМЕРТОНОСНЫЙ УДАР!")
26
27                foe_current_health -= damage
28                print(f"Вы наносите {damage} урона!")
29

```

Рисунок 5 – Листинг боевой системы

Создается противник с помощью класса `DungeonCreature`. Запускается цикл боя, который продолжается, пока здоровье игрока и противника больше нуля. На каждом ходу игрок выбирает действие: атака, использование предмета или попытка парирования.

При атаке учитывается шанс уклонения противника, который зависит от разницы в ловкости. Базовый урон определяется силой персонажа. Защита противника уменьшает получаемый урон. Гарантируется минимальный урон. Существует шанс критического удара в 10%, который удваивает наносимый урон.

После хода игрока противник контратакует с аналогичными расчетами. При победе игрок получает опыт, золото и с шансом 45% дополнительную добычу. Система обеспечивает тактическую глубину за счёт учёта множества факторов и предоставляет игроку возможность адаптировать свою стратегию под конкретного противника.

2.6 Система врагов

Класс `DungeonCreature` отвечает за создание и управление врагами в игре. Его структура представлена на рисунке 6.

```

1 class DungeonCreature:
2     def __init__(self, creature_level, current_chamber):
3         self.name = ''
4         self.max_hp = 0
5         self.current_hp = 0
6         self.brutality = 0
7         self.nimbleness = 0
8         self.defence = 0
9         self.depth_level = creature_level
10        self.chamber_number = current_chamber
11
12    def determine_creature_type(self):
13        fate_roll = randint(0, 100)
14
15        if 0 <= fate_roll < 10:
16            self.name = "Гоблинберсерк-"
17            self.max_hp = 32
18            self.brutality = 7
19            self.nimbleness = 2
20            self.defence = 2
21
22        elif 10 <= fate_roll < 20:
23            self.name = "Гномземлекоп-"
24            self.max_hp = 31
25            self.brutality = 5
26            self.nimbleness = 3
27            self.defence = 2
28
29        elif 95 <= fate_roll < 98:
30            self.name = "Каменный колосс"
31            self.max_hp = 79
32            self.brutality = 13
33            self.nimbleness = 1
34            self.defence = 4
35
36        else:
37            self.name = "ПОВЕЛИТЕЛЬ БЕЗДНЫ"
38            self.max_hp = 120
39            self.brutality = 8
40            self.nimbleness = 2
41            self.defence = 7
42
43        self.current_hp = self.max_hp
44

```

Рисунок 6 – Листинг класса врагов

Класс `DungeonCreature` содержит все характеристики врага: имя, здоровье, атаку, защиту, ловкость. Метод `determine_creature_type()` случайным образом выбирает тип врага из 12 возможных вариантов.

Каждый тип врага имеет уникальные базовые характеристики и вероятность появления. Более слабые враги, такие как Гоблин-берсерк и Гном-землекоп, имеют высокую вероятность появления (по 10% каждый). Средние по силе враги занимают основную часть вероятностного распределения.

Более сильные враги (Каменный колосс, Повелитель бездны) имеют меньшую вероятность появления, что делает их появление редким и значимым событием. Каменный колосс появляется с вероятностью 3%, а Повелитель бездны — всего 2%.

Враги адаптируются к уровню подземелья и количеству пройденных комнат через метод `adjust_to_depths()`. С каждым уровнем подземелья их характеристики увеличиваются, обеспечивая прогрессию сложности. Также учитывается количество пройденных комнат, что делает игру сложнее по мере продвижения игрока.

2.7 Система инвентаря и экипировки

Система инвентаря реализована через глобальный словарь `player_inventory` и методы управления предметами. Код системы приведён на рисунке 7.

```

1 player_inventory = {'Оружие': '', 'Доспех': ''}
2 for slot in range(1, 6):
3     player_inventory[str(slot)] = ''
4
5 def apply_gear_bonuses(self, gear_item):
6     bonus = self.extract_enhancement(gear_item)
7
8     if 'кинжол' in gear_item.lower() or 'меч' in gear_item.lower():
9         self.might += 3 + (bonus * 2)
10
11     elif 'кинжал' in gear_item.lower():
12         self.might += 1 + (bonus * 2)
13         self.finesse += 2 + (bonus * 2)
14
15     elif 'лук' in gear_item.lower():
16         self.finesse += 3 + (bonus * 2)
17
18     elif 'копье' in gear_item.lower():
19         self.might += 2 + (bonus * 2)
20         self.resilience += 1 + (bonus * 2)
21
22     elif 'доспех' in gear_item.lower():
23         self.resilience += 5 + bonus
24
25     elif 'рубаха' in gear_item.lower():
26         self.resilience += 3 + bonus
27         self.finesse += 1 + bonus
28

```

Рисунок 7 – Листинг системы инвентаря

Инвентарь реализован как словарь с ключами для оружия, брони и пяти слотов для предметов. Это обеспечивает простую и эффективную систему хранения и доступа к предметам.

Метод `apply_gear_bonuses()` анализирует название предмета и применяет соответствующие бонусы к характеристикам персонажа. Разные типы оружия дают различные комбинации бонусов, что позволяет игроку выбирать экипировку в соответствии со своим стилем игры.

Мечи увеличивают силу, что делает их эффективными против врагов с высокой защитой. Кинжалы дают бонусы к силе и ловкости, обеспечивая баланс между уроном и уклонением. Луки значительно увеличивают ловкость, что полезно для персонажей, полагающихся на критические удары и уклонения.

Копья повышают силу и стойкость, что делает их хорошим выбором для сбалансированных бойцов. Броня также имеет различия в предоставляемых бонусах: тяжёлые доспехи дают большой бонус к стойкости, а лёгкая броня обеспечивает баланс между защитой и подвижностью.

Метод `remove_gear_bonuses()` обеспечивает симметричное снятие бонусов при замене экипировки, что предотвращает накопление бонусов при частой смене предметов.

2.8 Система уровней и развития

Метод `gain_experience()` управляет повышением уровня персонажа. Код метода представлен на рисунке 8.

```
1 def gain_experience(self):  
2     if self.experience >= self.exp_to_next:  
3         self.level += 1  
4         self.experience -= self.exp_to_next  
5         self.exp_to_next = int(self.exp_to_next * 1.8)  
6         self.skill_points += 3  
7  
8         self.max_health += 12  
9         self.current_health += 12  
10        self.might += 2  
11  
12        return True  
13    return False  
14
```

Рисунок 8 – Листинг системы уровней

Проверяется, достиг ли накопленный опыт порога для повышения уровня. При повышении уровня увеличивается уровень персонажа. Вычитается опыт, необходимый для предыдущего уровня. Увеличивается требуемый опыт для следующего уровня по формуле геометрической прогрессии с множителем 1.8.

Добавляется 3 очка улучшения для ручного распределения между характеристиками. Это позволяет игроку кастомизировать своего персонажа в соответствии со своим стилем игры и текущими потребностями.

Автоматически увеличивается максимальное здоровье на 12 единиц, что обеспечивает постепенное усиление персонажа даже без ручного распределения очков. Текущее здоровье также восстанавливается на эту величину. Автоматически увеличивается сила на 2 единицы, что гарантирует рост боевой эффективности персонажа.

Игрок может использовать очки улучшения в комнатах отдыха для ручного распределения характеристик. Это создаёт стратегический элемент, так как игрок должен решать, какие характеристики улучшать в зависимости от текущей ситуации и своих предпочтений.

3 Тестирование

Для проверки корректности работы и сбалансированности игрового процесса было проведено ручное функциональное тестирование. Основные проверяемые сценарии включали:

- Создание персонажа: корректность генерации всех характеристик (здоровье, сила, ловкость, выносливость, рост, вес) в строго заданных для каждой расы диапазонах. Проверялась уникальность стартовой экипировки.

- Система развития: правильность начисления опыта за победу над врагами, своевременное повышение уровня персонажа и начисление очков улучшения. Проверялась работа функции ручного распределения этих очков.

- Экипировка и инвентарь: корректность применения бонусов от надетого оружия и брони к характеристикам персонажа. Тестировались операции добавления предметов в инвентарь, их использования и выбрасывания, а также обработка ситуации переполнения.

- Генерация подземелья: появление всех трёх заявленных типов комнат (бой, сокровище, отдых) с ожидаемой частотой. Проверялась логика выбора пути на развилках и увеличение уровня подземелья после заданного количества пройденных комнат.

- Боевая система: корректность расчёта урона с учётом атаки, защиты, минимального гарантированного урона и случайных факторов (критический удар, уклонение). Тестировались граничные случаи, такие как победа с минимальным здоровьем или поражение.

Все протестированные сценарии завершились успешно. Игра стабильно работает в консоли, не содержит критических ошибок или нарушений баланса, которые делали бы прохождение невозможным, и полностью соответствует исходному техническому заданию.

Заключение

В результате выполнения работы был успешно разработан полнофункциональный прототип консольной текстовой RPG. Программа реализует все ключевые механики, заявленные в задании: создание персонажа с системой рас, процедурную генерацию подземелья, тактическую пошаговую боевую систему, прогрессию через уровни и очки улучшения, а также систему инвентаря и экипировки.

Архитектура проекта, построенная на трёх основных модулях (`adventure.py`, `hero.py`, `dungeon_enemies.py`), обеспечивает чёткое разделение ответственности и хорошую расширяемость кода. Применение объектно-ориентированного подхода позволило создать логичные и переиспользуемые классы для персонажа и врагов.

Игра предоставляет законченный игровой цикл, сочетающий стратегическое планирование (развитие персонажа, выбор экипировки) с тактическими решениями (выбор пути, действия в бою) и элементом случайности, что соответствует канонам жанра roguelike. Проект демонстрирует практическое применение знаний, полученных в ходе изучения дисциплины «Информатика и программирование».